

Polynomial Ring 구현 및 Discrete Gaussian 조사

<https://youtu.be/-Z5PA5ZdjJk>
<https://youtu.be/dvuhjjYitJ4>

1. 한성대학교 To Do list

- Polynomial Ring 구현
 - Ring Dim = 256
 - $q > 2^{64}$
 - q in NTT friendly
- Discrete Gaussian 구현을 위한 관련 연구 및 Github 파악
 - Constant time 구현 적용 여부 및 성능 저하 분석
 - Sampler 구현을 위해서 convolution 기반의 구현 방법을 후보로 생각중
 - 논문 제목: Gaussian Sampling over the Integers: Efficient, Generic, Constant-time
 - link: <https://eprint.iacr.org/2017/259.pdf>
 - github: <https://github.com/malb/dgs>

2. polynomial Ring 구현

1단계: 프로토타입 (Python)

목표

정확성/구현 가능성 검증

구현

- 검증용 Schoolbook 구현
- $q > 2^{64}$ 에서의 NTT 동작

검증

NTT round-trip 검증
Schoolbook == NTT Mult ?



2단계: C레벨 이식 (C, u128)

목표

u128 자료형 구현 확인

구현

- __uint128_t 산술
- Barrett / Montgomery
- 모듈 분리

검증

Python 프로토타입과 값 비교
Schoolbook == NTT Mult ?



3단계 : 최적화 (NTT)

목표

성능 향상

구현

- 사전연산
- Lazy reduction + Radix-4
- Forward full-lazy

검증

2단계 구현과 결과 및 성능 비교

2. polynomial Ring 구현

❖ q 조건 검증

① $q > 2^{64}$

- 구현 목표

② $(q - 1) \bmod 2N = 0$

- NTT friendly 조건

❖ NTT 조건 검증

① $\psi^N \equiv -1$

- ψ = primitive $2N$ -th root of unity

② $\omega^{\frac{N}{2}} \equiv -1$

- ω = primitive N -th root of unity
- $(\psi^2 = \omega)$ 라서 ①이 만족하면 같이 자동 통과

t	Q	q	k	σ_s	σ_f	CPA	CCA		
						ctxt	tPKE	NIZK	ctxt
2	1	2^{28}	5	2^6	2^{17}	5.3	10.72	48.05	58.7
16	1	2^{50}	8	2^{12}	2^{29}	14.1	28.30	66.82	95.1
31	1	2^{29}	5	2^5	2^{19}	5.5	11.12	47.96	59.0
2	2^{32}	2^{45}	7	2^{11}	2^{33}	11.2	22.52	60.50	83.0
16	2^{32}	2^{66}	11	2^{17}	2^{45}	25.0	50.08	85.90	135.9
31	2^{32}	2^{46}	8	2^{10}	2^{36}	12.9	25.98	66.15	92.1
2	2^{60}	2^{59}	10	2^{16}	2^{48}	20.4	40.96	79.23	120.1
16	2^{60}	2^{81}	13	2^{22}	2^{60}	35.4	70.88	103.05	173.9
31	2^{60}	2^{60}	10	2^{15}	2^{50}	20.8	41.60	79.14	120.7

최대 q 는 2^{81}

2. polynomial Ring 구현

❖ 1단계: Python 프로토타입 구현

1) q 찾기

- 임의의 비트 수에서 조건 만족하는 prime q 검색

2) ψ 와 ω 조건 체크

- $\psi^N \equiv -1$

3) NTT 자체 검증

- round-trip 테스트로 검증
- Slow NTT == Cooley-Tukey NTT

4) 곱셈 검증

- Schoolbook(naïve) 검증
- $0 \cdot b == 0$
- $1 \cdot b == b$
- $X^{N-1} \cdot X == -1$ negacyclic 검증
- naïve == ntt (random 100쌍)

```
q = 664613997892457936451903530140225537 (bit length 120)
n = 256
psi = 646009819200409304245925530434338243
omega_psi^2 = 5118876012384836077108011895088251
q-1 / 2n factor = 1298074214633706907132624082305128

Property checks
PASS psi^(2n) == 1
PASS psi^n == -1 (q-1)
PASS omega^n == 1
PASS omega^(n/2) != 1
PASS INTT(NTT(a)) == a
PASS INTT_iter(NTT_iter(a)) == a
PASS iterative ntt_forward == O(n^2) ntt_forward
PASS iterative ntt_inverse == O(n^2) ntt_inverse

Multiplication tests (slow O(n^2) NTT)
PASS naive: 0 * b == 0
PASS ntt: 0 * b == 0
PASS naive: 1 * b == b
PASS ntt: 1 * b == b
PASS random 100: naive == ntt
```

2. polynomial Ring 구현

❖ 2단계: C레벨 이식

- C 레벨 이식에서의 문제
 - 일반적으로 컴퓨터의 레지스터는 32비트 또는 64비트
 - C언어에서 가장 큰 표준 공식 자료형은 64비트(long long)
- 64비트 시스템에서는 컴파일러 확장 기능으로 `__int128_t` / `__uint128_t` 자료형 제공
 - 64비트 레지스터 두 개를 조합하여 사용

```
u128 u128_add(u128 a, u128 b) { return a + b; }
```

`u128_add:`

```
add    rax, r8    ; lo + lo
adc     rdx, rdi    ; hi + hi + carry
ret
```

```
u64 u64_add(u64 a, u64 b) { return a + b; }
```

`u64_add:`

```
lea     rax, [rdi + rsi*1] ; rax = rdi + rsi
ret
```

2. polynomial Ring 구현



❖ 3단계: 최적화(NTT)

- NTT 연산에서 $x \bmod q$ 와 같은 modular 연산이 많이 사용됨

① 단순 %

- 나눗셈 연산이 포함
- CPU에서 느린 산술 연산

② Barrett

- $\mu = \lfloor R/q \rfloor$ 를 미리 계산
- mod 연산 $\rightarrow$ 곱셈 2번 + 뺄셈으로 대체

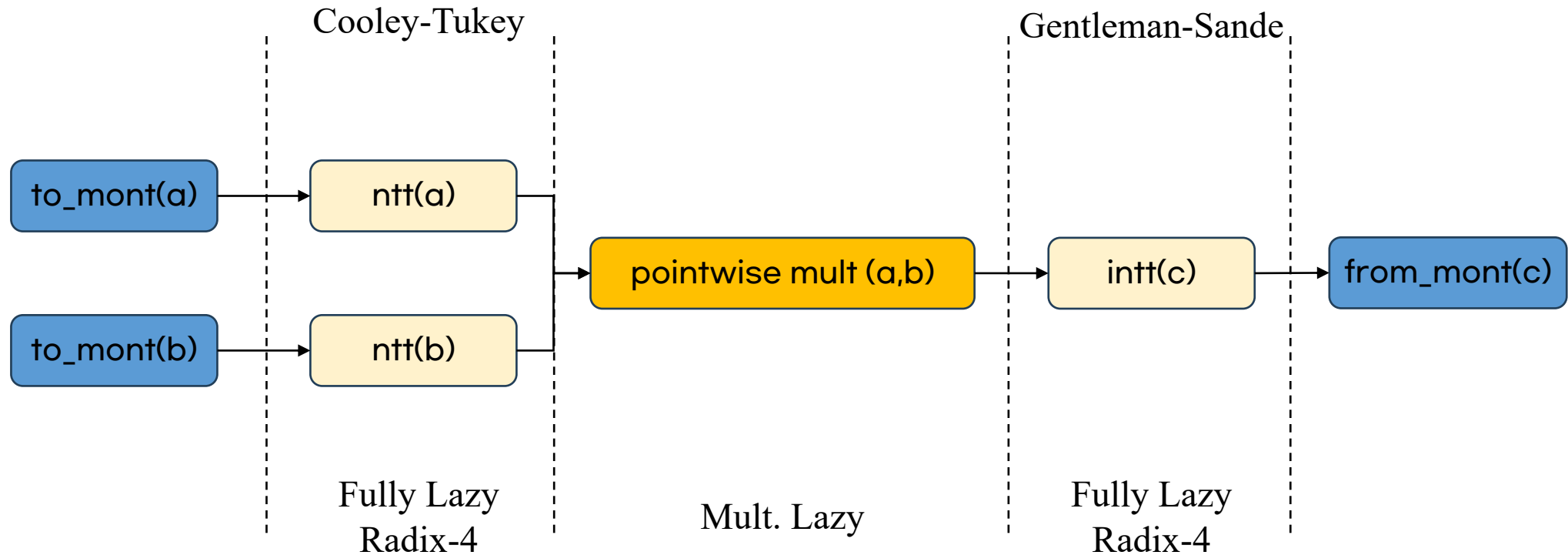
③ Montgomery

- 모든 값을 R배한 형태로 변환
- mod 연산 $\rightarrow$ 곱셈 1번과 REDC 1번으로 대체
- 대신, 변환 비용이 포함되지만 가려짐

2. polynomial Ring 구현

❖ 3단계: 최적화(NTT)

- 사전연산 + Montgomery + Radix-4 butterfly
 - 매 단계에서 필요한 $\text{to_mont}(\psi^k)$ 를 사전 연산하여 테이블 호출로 사용
 - Montgomery에서 Fully Lazy Reduction 적용
 - Radix-4는 레지스터 병렬이 아닌 반복문 안에서의 처리수를 의미



2. polynomial Ring 구현



❖ 3단계: 최적화(NTT)

- Unsigned 대신 Signed 자료형을 사용
- 다만, signed 를 적용하여 lazy 리덕션을 넣으려면 q 의 범위가 이론적으로 55비트까지만 사용 가능
- 마찬가지로 128비트 자료형으로 구현했을 때는 성능이 느림.
 - 그래서 q 가 64비트보다 크면 unsigned 가 좋음

2. polynomial Ring 구현

- 성능 측정

- $\mu\text{s/op}$, $N=256$
- 다항식 곱셈 60번 총 시간, 5번 반복 후 가장 빠른 값을 60으로 나눔

q.Bits	s64	u64	u128
2^{14}	3.07	5.47	31.23
2^{30}	3.07	5.47	31.18
2^{55}	-	5.47	31.11
2^{62}	-	-	31.12
2^{123}	-	-	31.06

						CPA	CCA		
t	Q	q	k	σ_s	σ_f	ctxt	tPKE	NIZK	ctxt
2	1	2^{28}	5	2^6	2^{17}	5.3	10.72	48.05	58.7
16	1	2^{50}	8	2^{12}	2^{29}	14.1	28.30	66.82	95.1
31	1	2^{29}	5	2^5	2^{19}	5.5	11.12	47.96	59.0
2	2^{32}	2^{45}	7	2^{11}	2^{33}	11.2	22.52	60.50	83.0
16	2^{32}	2^{66}	11	2^{17}	2^{45}	25.0	50.08	85.90	135.9
31	2^{32}	2^{46}	8	2^{10}	2^{36}	12.9	25.98	66.15	92.1
2	2^{60}	2^{59}	10	2^{16}	2^{48}	20.4	40.96	79.23	120.1
16	2^{60}	2^{81}	13	2^{22}	2^{60}	35.4	70.88	103.05	173.9
31	2^{60}	2^{60}	10	2^{15}	2^{50}	20.8	41.60	79.14	120.7

3. Discrete Gaussian Sampler



- DGS는 정수 집합 위에서 가우시안 분포를 따르는 값을 뽑는 과정
 - 일반적으로 가우시안분포는 실수 영역에서 정의됨
 - 암호 구현에서는 정수 계수나 다항식 계수로 사용해야 하므로 정수 위의 가우시안 분포가 필요
 - 격자기반 암호에서 Secret, error, noise 등을 특정 분포에서 샘플링함.
 - 분포가 정확하지 않으면 가정한 조건이 깨질 수 있고, 공격자가 분포의 편향을 이용하여 공격할 가능성이 있음
- 수학적 정의를 실제 컴퓨터에서 안전하게 구현하는 것이 어려움
 - Floating point의 오차 -> 플랫폼마다 float를 다루는 차이가 발생하여 플랫폼 차이도 고려
 - 실행 시간이 달라지며 발생하는 Timing 누출
 - 메모리 접근 패턴이 달라지면서 발생하는 Cache side channel 발생
 - 분포에 작은 편향이 생겨 발생할 수 있는 공격 가능성
- 구현 방식에 따라서 장단점이 달라짐
 - 대표적인 방법 : CDT, Knuth-Yao, Rejection, Bernoulli, Convolution Sampling

3. Discrete Gaussian Sampler



- 대표적인 구현 방법 비교

- CDT

- 샘플링 속도가 빠름, 사전 연산 테이블 사용
 - $\sigma \cdot \text{center}$ 에 선형 비례해 테이블 크기 증가 $\rightarrow$ 메모리 부담, 파라미터 바뀔 때마다 테이블 재생성

- Rejection Sampling

$$\exp(-x^2 / 2\sigma^2)$$

- 메모리 사용 적음, 임의의 σ 지원
 - Reject으로 인한 시간 변동, 부동소수점 연산 $\rightarrow$ 상수시간 구현 어려움

- Convolution Sampling

- 작은 σ 의 base sampler들을 합성해 큰 σ 생성
 - Sampler를 재사용해 다양한 σ 지원, 큰 σ 에서 메모리 효율적

t	Q	q	k	σ_s	σ_f	CPA	CCA		
						ctxt	tPKE	NIZK	ctxt
2	1	2^{28}	5	2^6	2^{17}	5.3	10.72	48.05	58.7
16	1	2^{50}	8	2^{12}	2^{29}	14.1	28.30	66.82	95.1
31	1	2^{29}	5	2^5	2^{19}	5.5	11.12	47.96	59.0
2	2^{32}	2^{45}	7	2^{11}	2^{33}	11.2	22.52	60.50	83.0
16	2^{32}	2^{66}	11	2^{17}	2^{45}	25.0	50.08	85.90	135.9
31	2^{32}	2^{46}	8	2^{10}	2^{36}	12.9	25.98	66.15	92.1
2	2^{60}	2^{59}	10	2^{16}	2^{48}	20.4	40.96	79.23	120.1
16	2^{60}	2^{81}	13	2^{22}	2^{60}	35.4	70.88	103.05	173.9
31	2^{60}	2^{60}	10	2^{15}	2^{50}	20.8	41.60	79.14	120.7

3. Discrete Gaussian Sampler



$$\bullet \sigma_{new}^2 = (x_1^2 + x_2^2) \cdot \sigma_{old}^2 \quad \rightarrow \quad \sigma_{new} = \sqrt{(x_1^2 + x_2^2)} \cdot \sigma_{old}$$

- $x_1 = x_2 = 2^k$ 일 때, $\sigma_{new} = \sqrt{2} \cdot 2^k \cdot \sigma_{old}$

- example

- $x_1 = x_2 = 2^{10}$ 일 때,

$$\sigma_{new} = \sqrt{(2^{20} + 2^{20})} \cdot \sigma_{old} = 2^{10.5} \cdot \sigma_{old}$$

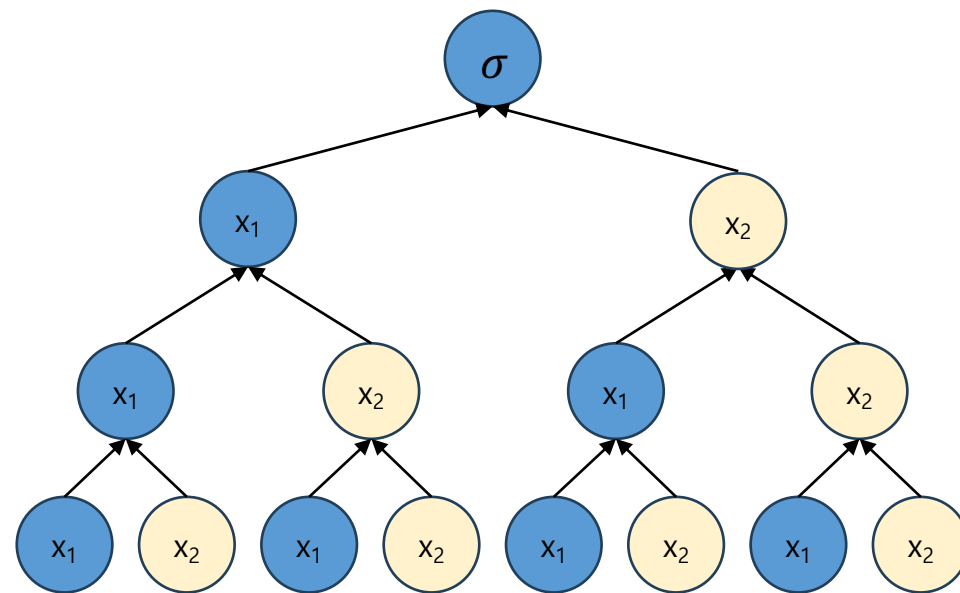
$$\sigma_{new} = \sqrt{(2^{20} + 2^{20})} \cdot \sigma_{old} = 2^{21} \cdot \sigma_{old}$$

$$\sigma_{new} = \sqrt{(2^{20} + 2^{20})} \cdot \sigma_{old} = 2^{31.5} \cdot \sigma_{old}$$

$$\sigma_{new} = \sqrt{(2^{20} + 2^{20})} \cdot \sigma_{old} = 2^{42} \cdot \sigma_{old}$$

$$\sigma_{new} = \sqrt{(2^{20} + 2^{20})} \cdot \sigma_{old} = 2^{52.5} \cdot \sigma_{old}$$

$$\sigma_{new} = \sqrt{(2^{20} + 2^{20})} \cdot \sigma_{old} = 2^{63} \cdot \sigma_{old}$$



3. Discrete Gaussian Sampler

- github: <https://github.com/malb/dgs>
 - 다양한 Discrete Gaussain Sampling 기법을 구현하고 비교할 수 있음
 - dp_Ver / mp_Ver - Floating point 기반
 - dp_Ver: double 자료형으로 부동소수점 연산을 사용.
 - mp_Ver: MPFR 기반의 임의 정밀도 부동소수점 연산을 사용. 더 높은 정밀도로 사용 가능

dp_Ver

σ	Rejection				CDT	Conv.
	online	table	logtable	$\sigma_2 + \text{logtable}$		
2^8	2.281	2.087	2.729	1.149	0.154	0.325
2^{16}	1.024	1.767	3.982	2.061	TIMEOUT	1.231
2^{30}	FAIL	OOM	FAIL	FAIL	OOM	0.606
2^{60}	FAIL	OOM	FAIL	FAIL	OOM	1.267

mp_Ver

σ	Rejection				CDT	Conv.
	online	table	logtable	$\sigma_2 + \text{logtable}$		
2^8	35.861	1.841	4.740	3.331	0.544	1.458
2^{16}	34.978	3.378	7.523	5.959	TIMEOUT	3.083
2^{30}	34.693	OOM	12.880	10.689	OOM	4.859
2^{60}	34.790	OOM	23.910	FAIL	OOM	10.424

online: 사전 표 없음

table: 사전 표 사용 + 메모리 사용 증가

logtable: 메모리 최적화

$\sigma_2 + \text{logtable}$: 가우시안 모양의 후보(reject이 줄어듦)

FAIL: σ 가 너무 크거나, double 정밀도 한계

OOM: overflow

Constant 구현 아님

3. Discrete Gaussian Sampler

- Gaussian Sampling over the Integers: Efficient, Generic, Constant-time
 - Convolution 최적화 constant time 구현 구조 제안
 - convolution은 기본적으로 작은 σ 를 합쳐서 목표 σ 를 샘플링함.

기존 방식 (dgs 라이브러리)

- σ 기반 동적 결정
 - 초기화 단계에서 σ 보고 Layer 수, 테이블 크기 결정
- 부동소수점
 - double, mpfr_t 사용
 - float 비교
- 입력에 의존적
 - 반복 횟수, 분기가 $\sigma \cdot \text{center}$ 에 따라 변함
- 따라서 실행시간 가변
 - 입력마다 실행 시간이 달라짐

논문 방식

- 상수로 고정
 - σ 과 무관, Layer 수와 크기를 미리 정해진 상수로 고정
- Fixed-point 정수
 - $P \times 2^{64}$ 미리 연산 $\rightarrow$ 테이블 lookup 사용
 - 매 호출은 정수 비트 비교
- 연산 과정 고정
 - 동일한 루프랑 테이블 조회를 고정
 - Mask 트릭으로 분기 제거
- Constant-time
 - 비밀 값과 관련된 값에 대해서는 constant
 - Base Sampler는 rejection 루프가 있음

3. Discrete Gaussian Sampler

- Gaussian Sampling over the Integers: Efficient, Generic, Constant-time
 - OpenFHE (<https://github.com/openfheorg/openfhe-development>)

Discrete Gaussian Generator Generic

- Implements the generic sampler by UCSD discussed in [Gaussian Sampling over the Integers: Efficient, Generic, Constant-Time](https://eprint.iacr.org/2017/259.pdf)

```
/*This is the header file for the Generic Sampler used for various Discrete
 * Gaussian Sampling applications. This class implements the generic sampler by
 * UCSD discussed in the https://eprint.iacr.org/2017/259.pdf and it is heavily
 * based on Michael Walter's original code. Along the sides of the
 * implementation there are also two different "base samplers", which are used
 * for the generic sampler or can be used on their own depending on the
 * requirements of needed application.
```

```
// OpenFHE default
#define MAX_LEVELS 4

// Test override
#define MAX_LEVELS 5
```

level 4 : wide $\sigma \approx 2^{22}$
level 5 : wide $\sigma \approx 2^{62}$

```
// OpenFHE default
return x1 * sampler1->Gen..() + x2 * sampler2->Gen..();

// Test override
return static_cast<__int128>(x1) * sampler1->Gen..() +
       static_cast<__int128>(x2) * sampler2->Gen..();
```

큰 σ 에서 발생하는
Convolution 중간값
overflow로 인해.

3. Discrete Gaussian Sampler

- DGS 라이브러리와 openFHE-Constant Covolution 구현 비교

- ❖ dgs 라이브러리

σ 기반 동적 결정

dgs_gauss_dp.c

```
//  $\sigma$ 가 작아질 때까지 while  $\rightarrow$  recursion_level이  $\sigma$ 로 결정됨
while (table_size > MAX_TABLE_SIZE_BYTES) {
    recursion_level++;
    z1 = floor(sqrt(current_sigma / (eta * 2)));
    z2 = (z1 > 1) ? z1 - 1 : 1;
    current_sigma /= sqrt(z1*z1 + z2*z2);
    table_size = 2 * ceil(current_sigma * tau) * per_entry;
}
n_coefficients = 1 << recursion_level; //  $\sigma$  의존
```

- ❖ openFHE 라이브러리

상수 고정

discretegaussianguardergeneric.cpp

```
// 컴파일 타임 상수.  $\sigma$ 와 완전 무관.
#define MAX_LEVELS 5 // header

// 항상 4번 반복 ( $\sigma$ 가  $2^{30}$ 이든  $2^{60}$ 이든 동일)
for (int i = 1; i < MAX_LEVELS; ++i) {
    x1 = floor(sqrt(wide_variance / (2.0*N*N)));
    x2 = max(x1 - 1, 1);
    wide_sampler = new SamplerCombiner(...);
}
k = ceil((PRECISION - BERNOULLI_FLIPS) / log_base); // = 6, 고정
```

3. Discrete Gaussian Sampler

• DGS 라이브러리와 openFHE-Constant Covolution 구현 비교

❖ dgs 라이브러리

부동소수점 + float 비교

dgs_gauss_dp.c

```
// y, z는 double — 부동소수점 계산
do {
  x = self->c_z + _dgs_randomm_libc(...) - ...;
  z = exp(((double)x - c)*((double)x - c) * self->f);
  y = drand48();           // double [0,1)
} while (y >= z);          // float 비교 (CPU·값 따라 시간 다름)
```

❖ openFHE 라이브러리(Knuth_yao 방식)

PRNG 1비트씩 트리 하강

Fixed-point 정수

discretegaussiangueratorgeneric.cpp

```
// (init) 확률을 2^64 정수로 환산해 테이블에
probMatrix[i] = probs[i] * (1.0/S) * std::pow(2, 64);           // uint64
hammingWeights[j] += ((probMatrix[i] >> (63-j)) & 1);

// (call) PRNG 1비트 + 정수 시프트 + 정수 비교만
bit = bg->Generate();
nodeIndex = nodeIndex * 2 + bit;
if (ans >= 0 && ans != b_matrixSize - 1) hit = true;
```

3. Discrete Gaussian Sampler

• DGS 라이브러리와 openFHE-Constant Covolution 구현 비교

❖ dgs 라이브러리

가변 흐름 - rejection + if 분기

dgs_gauss_dp.c

```
// 반복 횟수가 입력 의존
do {
    x = ...;
    z = ...;
    y = drand48();
}
while (y >= z);          // 횟수 가변 → 시간 가변

// center에 따라 if 분기
if (self->coset_sampler) { // 분수 center일 때만 호출
    x += self->coset_sampler->call(...);
}
```

❖ openFHE 라이브러리(Knuth_yao 방식)

PRNG 1비트씩 트리 하강

고정 흐름 + Mask 트릭

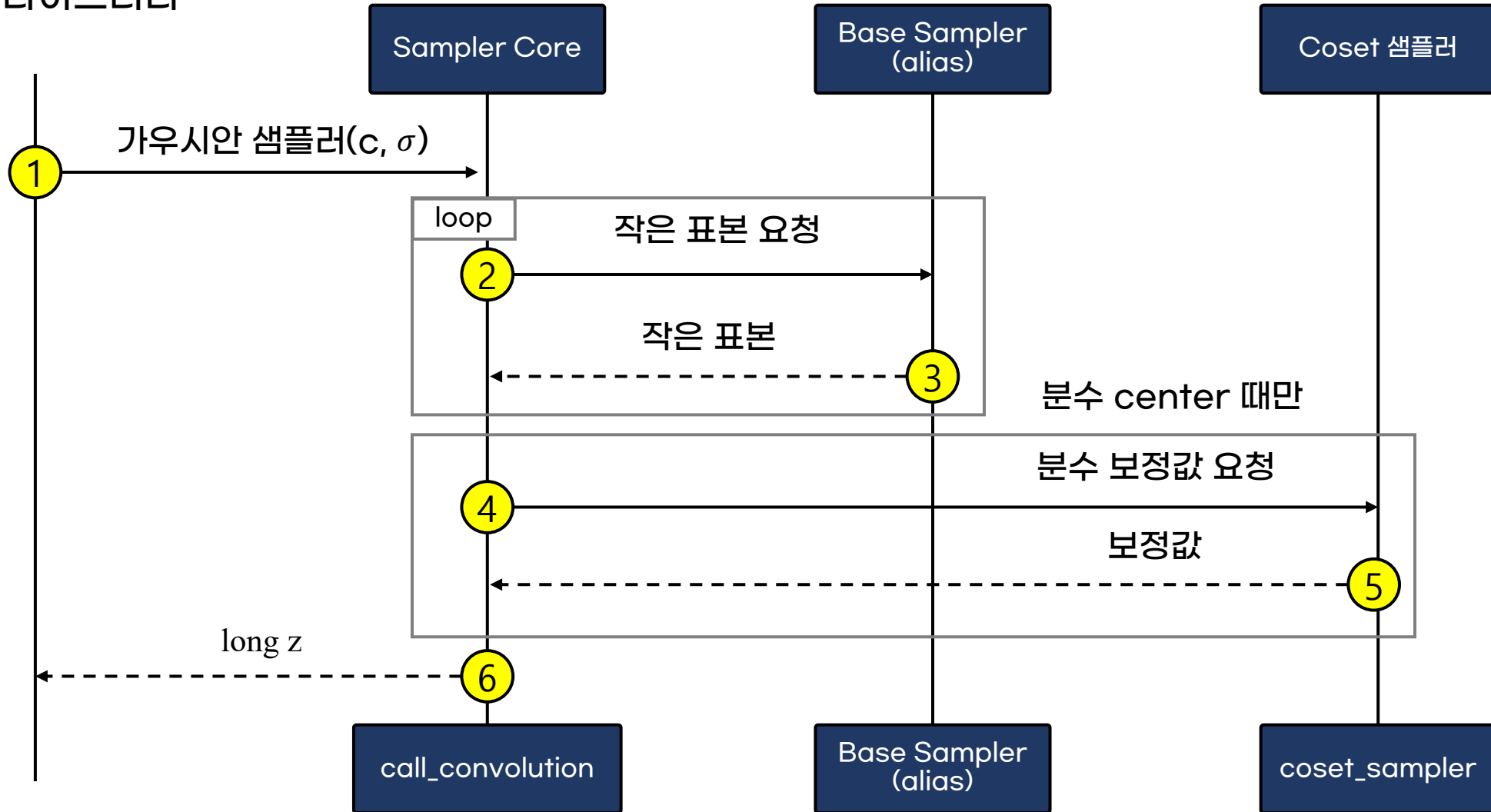
discretegaussiangeneratorgeneric.cpp

```
// 항상 k=6번. 분기 대신 mask 인덱싱으로 sampler 선택
for (int i = 0; i < k; ++i) {
    // "mask & c"의 값(0~31)이 곧 sampler 인덱스 — if 문 없음
    sample = base_samplers[mask & c]->GenerateInteger();
    c >>= log_base; c += sample;
}

// flipAndRound — 23비트 모두 결정론적 비교
for (int i = BERNOULLI_FLIPS-1; i >= 0; --i) {
    rb = base_samplers[0]->RandomBit();
    if (rb > extractBit(c, i)) return SampleC(base_c);
    if (rb < extractBit(c, i)) return SampleC(base_c + 1);
}
```

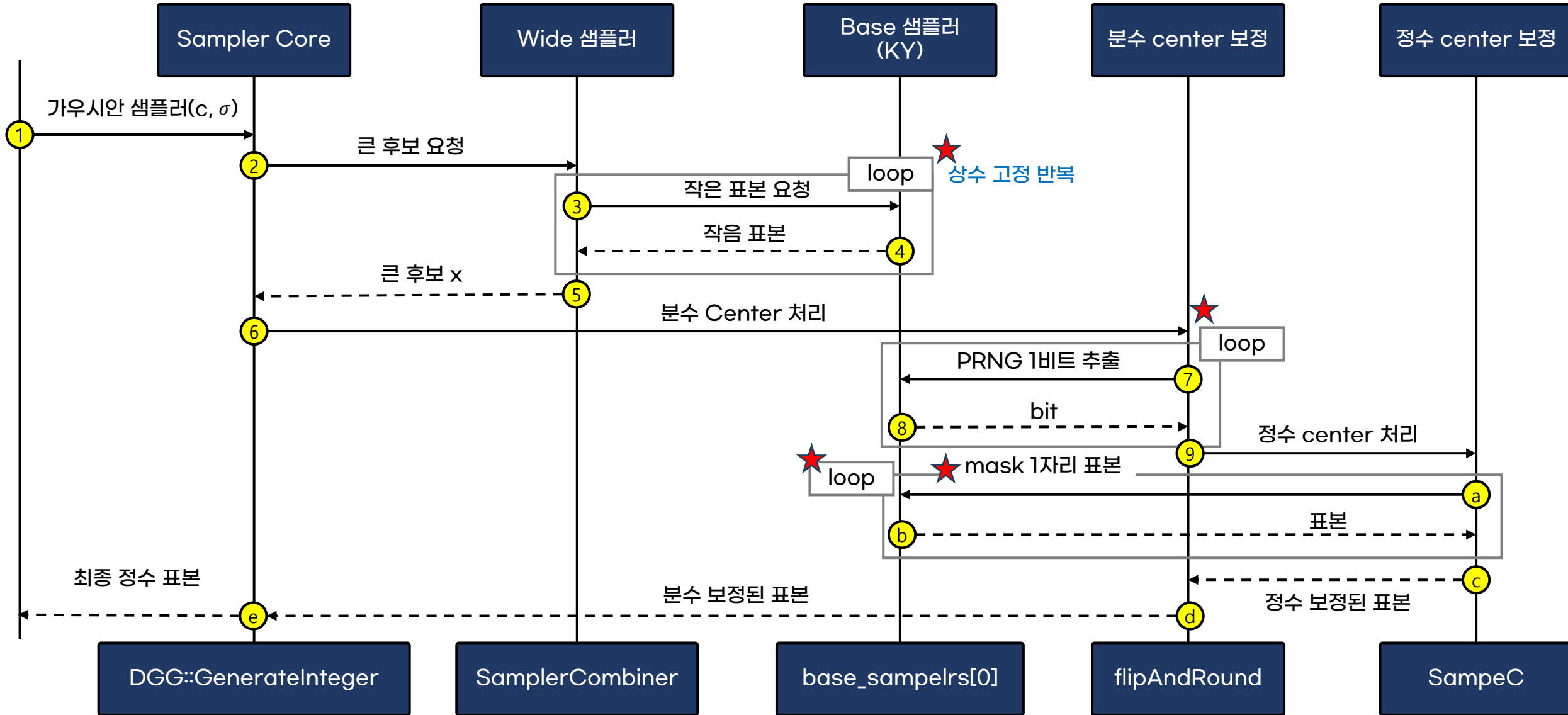
3. Discrete Gaussian Sampler

❖ dgs 라이브러리



3. Discrete Gaussian Sampler

❖ openFHE 라이브러리



3. Discrete Gaussian Sampler

- 전체적인 Discrete Gaussian Sampler 성능 비교

σ	dgs		openFHE(KY)		openFHE(Peikert)	
	성능(μs)	메모리	성능(μs)	메모리	성능(μs)	메모리
2^8	0.171	≤ 200 KB	0.697	19 MB	0.958	360 KB
2^{16}	0.754	≤ 200 KB	0.694	19 MB	0.959	360 KB
2^{30}	0.739	≤ 200 KB	0.687	19 MB	0.955	360 KB
2^{60}	1.371	≤ 200 KB	0.680	19 MB	0.956	360 KB

4. 논의 사항 및 추후 진행

- 성능을 위해 보안 등급에 따라 다른 구현을 활용해야 하는지?
 - s64, u64, u128
- Discrete Gaussian Constant 구현 및 성능 비교
 - PRNG 함수에 AES-CTR 적용
 - 현재 C++ std:: 제공 PRNG 함수 사용
 - Knuth-Yao와 Peikert 비교

```
class PseudoRandomNumberGenerator {  
public:  
    static std::mt19937& GetPRNG() {  
        thread_local static std::mt19937 prng{std::random_device{}()};  
        return prng;  
    }  
};
```

감 사 합 니 다